



# Attachment canvas

Preview heterogeneous content through one typed surface.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-attachment-canvas



# When this library is useful

Preview heterogeneous content through one typed surface.

01

## Inspect evidence

Open a cited PDF or uploaded text beside a conversation.

02

## Choose a renderer

A content discriminator selects the appropriate view.

03

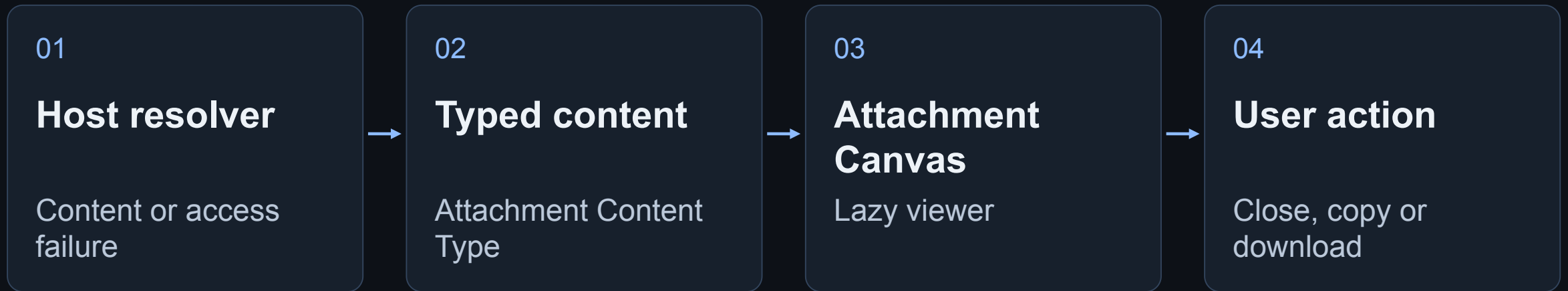
## Extend a workspace

Add visualizers or MCP content through explicit inputs.



# Follow the ownership boundary

Inputs and effects stay explicit.



Library: Viewer selection, panel layout, lazy renderer loading and optional canvas context.



# The public contract to start with

AttachmentCanvas · selected surface; full API index is in the bundle.

| Input or operation | Contract                |
|--------------------|-------------------------|
| content            | AttachmentCanvasContent |
| isOpen             | boolean                 |
| onClose            | () => void              |
| labels             | AttachmentCanvasLabels  |



# A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
<AttachmentCanvas
  isOpen={open}
  onClose={() => setOpen(false)}
  content={{
    type: AttachmentContentType.PlainText,
    text: 'Demo evidence',
  }}
  labels={{ ariaLabel: 'Demo preview' }}
/>
```

Full example: [examples/attachment-canvas.tsx](#) · uses [@epam/ai-dial-attachment-canvas](#)



# Wire a realistic host workflow

Keep attachment canvas focused on its public responsibility.

01

## Resolve a file

Fetch accessible content in the host and choose its content kind.

02

## Open the preview

Supply content, open state, labels and any renderer setup.

03

## Handle an action

Close locally or download through a host callback.



# Context-based canvas control

AttachmentCanvasProvider owns shared canvas state.

```
import { AttachmentCanvasProvider }  
  from '@epam/ai-dial-attachment-canvas';  
  
<AttachmentCanvasProvider>  
  {children}  
</AttachmentCanvasProvider>
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



# Available customization

01

## Typed content variants

Plain text, code, Markdown, PDF, OOXML, media, HTML and MCP views.

02

## Optional shared context

A canvas provider and its context hook coordinate shared preview state.

03

## Host loading hooks

loadPdf and configurePdfWorker support caller-controlled PDF setup.



# Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

| Evidence      | Where to look                                                                      |
|---------------|------------------------------------------------------------------------------------|
| Real consumer | mcp-apps/src/hooks/useMcpAppInlinePreview/<br>useMcpAppInlinePreview.ts            |
| Test evidence | routes an OOXML attachment by extension when its MIME type is generic              |
| Test evidence | renders plain text immediately with no syntax highlighter for a plaintext language |



# Constraints that affect integration

01

## Heavy renderers

PDF, Office, syntax and MCP dependencies affect packaging; several load lazily.

02

## Content errors differ

Unsupported, forbidden and failed loads have distinct display states.

03

## Untrusted content

HTML and MCP views require the documented sandbox and host settings.